

Assignment 5: Agentic AI with RAG

Assignment: Build an Agentic AI Research Assistant with RAG + Tools (AutoGen + Local LLM)

“Agentic Academic Assistant: Retrieval, Reasoning, and Tool Use with Local LLMs”

Assignment Objectives:

Build a multi-agent system using AutoGen that can:

Retrieve relevant information using RAG

Use functions/tools for structured tasks

Run on a local open-source LLM (LM Studio)

Produce a well-structured answer with citations

Problem Scenario

You are building an AI assistant for university students that can answer questions like:

“Explain Retrieval-Augmented Generation and compare it with fine-tuning. Include examples and references.”

The assistant must:

Retrieve relevant content from a local knowledge base (PDFs / text files) using RAG

Use tools/functions when needed

Produce structured, grounded answers (no hallucination)

System Requirements

1. Multi-Agent Setup (AutoGen)

You must implement at least 3 agents:

1. Planner Agent

Breaks the user query into steps

Decides:

When to retrieve

When to call tools

2. Retriever Agent

Uses RAG pipeline

Fetches relevant chunks from knowledge base

3. Answering Agent

Synthesizes final answer

Adds citations from retrieved context

Knowledge Base (RAG)

Use any of the following:

Lecture notes (your course content)

Research papers (PDFs)

Articles on:

LLMs

RAG

Fine-tuning

Agentic AI

You should use at least 20 documents related to above topics.

Minimum Requirements:

You must use:

Chunking

Embeddings (e.g., sentence-transformers)

Vector DB (e.g., ChromaDB)

Tools / Function Calling (MANDATORY)

You must implement at least 2 tools:

Example Tool 1: Citation Formatter

```
def format_citation(text: str, source: str) -> str:
```

```
    return f"{text} (Source: {source})"
```

Example Tool 2: Word Count / Summarizer

```
def summarize_text(text: str, max_words: int) -> str:
```

```
    # basic truncation or logic
```

Agents must decide when to call these tools

LLM Requirement (IMPORTANT)

You must use a local open-source LLM via LM Studio, such as:

Phi-2

Mistral-7B-Instruct

LLaMA 3

Expected Workflow

User asks a question

Planner Agent:

Breaks task

Calls Retriever

Retriever:

Returns top-k chunks

Answer Agent:

Uses retrieved context

Calls tools (if needed)

Generates structured answer

Input / Output Example

Input:

“Compare RAG and fine-tuning in LLMs with examples.”

Output (Expected):

Structured answer:

Definition of RAG

Definition of Fine-tuning

Comparison table

Example use cases

Citations from retrieved documents

Submission Guideline:

Submit to the link, either ipynb or .py file. Do not submit other files which you keep in RAG, the teacher will ask you to present the work using your laptop so keep your environment ready.